ADAPTIVE QUESTION PAPER GENERATOR FOR STUDENTS

M.Tech (Integrated)

in

School of Computer Science and Engineering

With

Specialization in <To be filled>

By

<Reg No > <Name>

For

CSI4004 TEXT MINING

Under the guidance of

Dr. Lokesh Kumar R.

School of Computer Science and Engineering



VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

November, 2026

TABLE OF CONTENTS

CHAPTER	TITLE		PAGE
	LIST OF FIGURES		**4**
	LIST OF ABBREVIATIONS		**5**
	LIST OF TABLES		**6**
1	**ABSTRACT**		**7**
2	**INTRODUCTION**		**8**
	2.1	BACKGROUND	8
	2.2	OBJECTIVES	8
	2.3	APPLICATIONS	9
	2.4	SCOPE AND LIMITATIONS	9
3	**LITERATURE SURVEY**		**11**
	3.1	ADAPTIVE LEARNING	11
	3.2	LEARNER MODELLING	11
	3.3	TEST ASSEMBLY AND MARKS	11
	3.4	AUTOMATIC QUESTION GENERATION	12
	3.5	SUMMARY OF THE LITERATURE	12
4	**PROBLEM STATEMENT**		**13**
	4.1	PROBLEM DEFINITION	13
	4.2	RESEARCH GAP	13
	4.3	INPUTS, PROCESSING AND OUTPUTS	14
	4.4	SUCCESS CRITERIA	14
5	**METHODOLOGY**		**16**
	5.1	SYSTEM ARCHITECTURE	16
	5.2	TOOLS AND TECHNOLOGIES	17
	5.3	ADAPTIVE MARK ALLOCATION	18
	5.4	QUESTION SELECTION	19
	5.5	PERFORMANCE TRACKING	20
	5.6	AI QUESTION GENERATION	21
	5.7	DATABASE DESIGN	21
	5.8	USER INTERFACE DESIGN	22
	5.9	TESTING METHOD	23
6	**RESULTS**		**24**

	6.1	ADAPTIVE MARK ALLOCATION	24
	6.2	MASTERY TRACKING	24
	6.3	PREVIOUSLY SOLVED QUESTIONS	25
	6.4	AI PAPER GENERATION	25
	6.5	AUTOMATED TESTING	26
	6.6	USER INTERFACE	27
	6.7	DISCUSSION	30
7	**FUTURE SCOPE**		**32**
	7.1	IMPROVEMENTS	32
	7.2	RESEARCH DIRECTIONS	32
8	**REFERENCES**		**33**
	8.1	LIST OF JOURNALS	33
	8.2	LIST OF WEBSITES (URLs)	33
	8.3	LIST OF BOOKS	33

LIST OF FIGURES

SR. NO.	NAME OF THE FIGURE	PAGE NO.
1	Figure 5.1: System architecture of the Adaptive Question Paper Generator	16
2	Figure 5.2: Closed feedback loop between attempts and the next paper	17
3	Figure 5.3: Flowchart of curriculum paper generation	20
4	Figure 5.4: Entity-relationship diagram of the database	22
5	Figure 6.1: Marks given to each chapter before and after the first attempt	24
6	Figure 6.2: Mastery score compared with the score in each attempt	25
7	Figure 6.3: Login page	27
8	Figure 6.4: Student dashboard	28
9	Figure 6.5: Make a practice paper (generate page)	28
10	Figure 6.6: Question paper with "Previously solved" tags	29
11	Figure 6.7: Attempt page with self-assessment	29
12	Figure 6.8: Results page with chapter scores and solutions	30
13	Figure 6.9: Paper written by the AI mode	30

LIST OF ABBREVIATIONS

SR.NO.	ABBREVIATION
1	AI - Artificial Intelligence
2	API - Application Programming Interface
3	AQG - Automatic Question Generation
4	BKT - Bayesian Knowledge Tracing
5	CBSE - Central Board of Secondary Education
6	CSS - Cascading Style Sheets
7	DB - Database
8	EMA - Exponential Moving Average
9	ER - Entity-Relationship
10	HTML - HyperText Markup Language
11	HTTP - HyperText Transfer Protocol
12	IRT - Item Response Theory
13	JSON - JavaScript Object Notation
14	LA - Long Answer
15	LLM - Large Language Model
16	MCQ - Multiple Choice Question
17	OCR - Optical Character Recognition
18	ORM - Object-Relational Mapping
19	PDF - Portable Document Format
20	SA - Short Answer
21	SQL - Structured Query Language
22	UI - User Interface
23	URL - Uniform Resource Locator
24	UX - User Experience

LIST OF TABLES

SR.NO.	NAME OF THE TABLE	PAGE NO.
1	Table 3.1: Summary of the reviewed literature and its use in this project	12
2	Table 4.1: Inputs, processing and outputs of the system	14
3	Table 4.2: Comparison of the existing approach and the proposed system	14
4	Table 5.1: Software components and their purpose	18
5	Table 5.2: Chapter classification and weights	19
6	Table 5.3: Scores used to rank candidate questions	19
7	Table 5.4: Main routes of the web application	23
8	Table 5.5: Tables of the database	22
9	Table 6.1: Mark allocation in a 30-mark paper	24
10	Table 6.2: Mastery after four attempts	25
11	Table 6.3: AI paper generation: one request per question and one request per paper	26
12	Table 6.4: Automated tests	27

CHAPTER 1

ABSTRACT

Students preparing for the Class 10 and Class 12 board examinations usually practise from static question papers that give every student the same mix of questions, whatever they already know. This project presents an Adaptive Question Paper Generator, a web application that builds a board-pattern practice paper for each student and uses the student's own attempt history to decide how many marks every chapter receives.

After each attempt the student enters question-wise marks. The system converts them into chapter percentages, updates a smoothed mastery score (0.6 x latest + 0.4 x previous), and classifies every chapter as Weak (below 40%), Average (40% to 74%) or Strong (75% and above). Weights of 1.8, 1.2, 0.7 and 1.0 are given to Weak, Average, Strong and unattempted chapters. The Largest Remainder (Hare-Niemeyer) method turns the weights into whole marks that add up exactly to the requested total, and a backtracking subset-sum search selects questions that meet those marks. Questions the student has already solved in an earlier paper are tagged "Previously solved".

An optional AI mode uses the Google Gemini API to write new questions with a single request per paper and falls back to the question bank when the AI is unavailable. The application is written in Python with Flask, SQLite and ReportLab, exports papers as PDF, and is checked by 27 automated tests. In a worked example, a chapter in which the student scored 20% received 14 of 30 marks, while a chapter scored at 100% received 6. Because the core logic is rule-based and deterministic, every mark allocation can be traced to a stated rule.

Keywords: Adaptive learning, question paper generation, mastery tracking, Largest Remainder method, subset-sum selection, large language model, Flask

CHAPTER 2
INTRODUCTION

2.1 BACKGROUND

Board examination preparation depends heavily on practice. Most students practise from textbooks, guidebooks and previous-year papers, in which every student receives the same questions in the same proportions. A student who is already confident in Statistics but weak in Trigonometry therefore spends as much time on Statistics as on Trigonometry. A teacher who wants to personalise the practice has to read the past scores, estimate the weak chapters, pick questions, balance the marks across sections and format the paper by hand, which is slow and does not scale beyond a few students.

The Adaptive Question Paper Generator is a full-stack web application written in Python that automates this workflow. A student signs in, chooses a subject, the chapters to practise, the total marks and the difficulty, and the system builds a practice paper in the board pattern (Section A: 1-mark questions, Section B: 2-mark, Section C: 3-mark, Section D: 4 to 5 mark). The student attempts the paper, enters the marks obtained for each question, and the system updates its picture of the student. The next paper then gives more marks to the chapters in which the student is weaker and fewer, but never zero, to the chapters already mastered.

The application stores its data in SQLite and holds a question bank of 74 CBSE-style questions covering three subjects (Class 10 Mathematics, Class 10 Science and Class 12 Mathematics) and seven chapters. Papers can be shown on screen, downloaded as a PDF, and reviewed afterwards with step-by-step solutions. An optional AI mode can ask a language model to write fresh questions. The central decisions of the system, however, are made by simple rules that are deterministic and can be explained to a student or a teacher.

Although the adaptive part is rule-based, the project handles text at several points, which connects it to the text mining course. Question statements, options, answers and marking schemes are stored and processed as text; replies from the language model are cleaned of markdown fences and parsed as JSON, including the repair of LaTeX backslashes that break JSON; unusable replies are detected and replaced; and mathematical text is rendered with MathJax.

2.2 OBJECTIVES

The objective of the project is to build a web application that automatically generates a board-pattern practice paper for each student, in which the distribution of marks across chapters is driven by that student's own attempt history, and that updates its understanding of the student after every attempt. The specific aims are:

- to generate a paper whose total is exactly the requested number of marks, in the Section

A to D board pattern;

- to classify every chapter as Weak, Average or Strong from the attempt data and to give weak chapters a larger share of marks than strong chapters, while keeping strong chapters in the paper for revision;
- to update a smoothed mastery value after each attempt so that one bad or lucky attempt does not swing the next paper too far;
- to keep the core logic deterministic, explainable and covered by automated tests;
- to tell the student clearly which questions were already solved before ("Previously solved" tag);
- to give instant feedback with solutions, a topic-wise report and a printable PDF;
- to offer an optional AI mode that writes new questions and falls back safely when the AI is not available;
- to present all this in a simple interface that a student of about fifteen years can understand without help.

2.3 APPLICATIONS

- **Self-study by students:** regular practice papers that automatically concentrate on the weakest chapters before the board examinations.
- **Teachers and tutors:** personalised papers for a whole class in seconds, with a data-backed view of each student's weak chapters.
- **Schools and coaching centres:** consistent paper quality and format across batches, and a question bank that can grow chapter by chapter.
- **Parents:** chapter-level progress instead of a single overall score.
- **Other subjects and boards:** the engine works on any set of chapters and questions with marks, so more subjects can be added by adding rows to the question bank.
- **Low-resource classrooms:** the core engine needs no cloud service, so it can run on an ordinary computer on a local network.

2.4 SCOPE AND LIMITATIONS

The system covers paper generation, attempt entry, performance analysis and PDF export for Class 10 and Class 12. It has the following limitations:

- The question bank is small (74 questions; Class 10 Science has only four questions per chapter), so repeats are likely after a few papers.
- Marks are entered by the student, so the accuracy of the mastery score depends on honest and correct self-marking.
- The effect on learning outcomes has not yet been measured in a real classroom trial.

- AI-written questions are not reviewed by a teacher. On the free tier of the Gemini API the model used in the tests allowed only 20 requests per day, so the number of AI papers per day is limited.
- The sign-in with Google is of demonstration grade: the token is decoded but its signature is not verified. It must be hardened before any public deployment.
- Questions are text only; diagrams are not supported. The interface loads Bootstrap, Chart.js, MathJax and fonts from content delivery networks, so the full interface needs an internet connection.

CHAPTER 3
LITERATURE SURVEY

This section reviews the work that the project builds on: adaptive learning, learner modelling, the assembly of tests from a question bank, the sharing of marks, and the automatic generation of questions. Table 3.1 at the end of the section shows how each source is used.

3.1 ADAPTIVE LEARNING

An adaptive system changes what it shows to a user according to a model of that user. Brusilovsky [1] surveyed the methods and techniques of adaptive hypermedia and showed that a user model, built from what the user has done and knows, is the basis on which content and navigation are adapted. In education this idea leads to adaptive practice: the material that a student receives depends on what the student already knows. The present project applies the same idea at the level of a whole question paper. The user model is the mastery of every chapter, and the adaptation is the share of marks that each chapter receives.

3.2 LEARNER MODELLING

Corbett and Anderson [2] introduced knowledge tracing, which estimates the probability that a student knows a skill and updates the estimate after every answer; the method has been used in intelligent tutoring systems. Pelanek [3] reviews learner-modelling techniques, such as Bayesian knowledge tracing and logistic models, and discusses how they differ in complexity and in how easily they can be interpreted. Such models need response-level data and parameters that are estimated from many students.

A new system that has no such data can start with a simpler estimate. Exponential smoothing [12] gives more weight to recent observations by a fixed factor, needs no training data and is easy to explain to a student. This project uses exponential smoothing with a factor of 0.6 for the mastery score of every chapter, and keeps Bayesian knowledge tracing and Item Response Theory as future work (Section 7).

3.3 TEST ASSEMBLY AND MARKS

van der Linden [14] treats the assembly of a test from an item bank as an optimisation problem with constraints on content, length and difficulty. Selecting questions so that their marks add up to an exact total is a subset-sum problem, which is NP-complete in general [11] but easy to solve by backtracking at the size of a school question bank.

Sharing a total among chapters in proportion to weights is an apportionment problem, as in the allocation of seats in proportional representation. The Largest Remainder (Hare-Niemeyer) method [10] gives whole numbers that add up exactly to the total, which is what an examination

paper needs.

3.4 AUTOMATIC QUESTION GENERATION

Kurdi et al. [4] reviewed research on the automatic generation of questions for educational purposes, including how questions are produced and how their quality is evaluated. Large language models such as Gemini [8] can now write examination-style questions from a short instruction, but the output can be wrong or badly formed. The project therefore validates every reply, replaces unusable questions from the question bank, falls back to the bank when the AI is not available, and states that AI-written questions need teacher review.

3.5 SUMMARY OF THE LITERATURE

Table 3.1: Summary of the reviewed literature and its use in this project

Source	Main idea	Use in this project
Brusilovsky [1]	Adaptation depends on a model of the user	Chapter mastery is the student model
Corbett and Anderson [2]; Pelanek [3]	Probabilistic learner models need response data	Kept as future work; a simple smoothed score is used now
Hyndman and Athanasopoulos [12]	Exponential smoothing weights recent data more	Mastery = 0.6 x latest + 0.4 x previous
van der Linden [14]; Cormen et al. [11]	Tests are assembled under constraints; subset-sum	Backtracking subset-sum picks questions for exact marks
Largest remainder method [10]	Whole-number apportionment with an exact total	Marks per chapter add up exactly to the total
Kurdi et al. [4]; Gemini [8]	Questions can be generated automatically; quality must be checked	Optional AI mode with validation and fallback

CHAPTER 4
PROBLEM STATEMENT

4.1 PROBLEM DEFINITION

Students preparing for the Class 10 and Class 12 board examinations practise from static, generic question papers. Every student receives the same mix of questions, whatever they already know, so time is spent re-solving chapters that are already mastered while weak chapters receive no extra attention. Teachers and tutors who try to personalise the papers must do it by hand: they analyse past scores, pick questions, balance marks across sections and format the paper. This is slow and inconsistent and does not scale beyond a few students.

The problem addressed by this project is therefore: *how can a board-pattern practice paper be generated automatically for each student, so that the marks given to each chapter follow the student's own weaknesses, the total is exactly the requested number of marks, and the system learns from every attempt?*

4.2 RESEARCH GAP

The literature of Section 3 shows several useful ideas but no single tool that meets the need of a school student for board-pattern papers. Online test platforms deliver fixed tests with an overall score. Probabilistic learner models need response data and are harder to explain. Language models can write questions, but they do not guarantee a paper with an exact total or a board layout, and their output must be validated. The gap is a system that combines an explainable chapter model, exact mark allocation, exact-total question selection in the board pattern, visible handling of repeated questions and a safe, optional use of AI. Table 4.2 compares the existing approach with the system proposed here.

Table 4.2: Comparison of the existing approach and the proposed system

Aspect	Existing approach	Proposed system
Question mix	Same for every student	Built for each student from the attempt history
Chapter emphasis	Teacher's judgement	Weights 1.8 / 1.2 / 0.7 / 1.0 from the chapter level
Total marks	Counted by hand, often approximate	Always exact, by Largest Remainder and subset-sum search
Repeated questions	Not tracked	Tagged "Previously solved" with date and score
Feedback	Overall score	Chapter-wise report with solutions and next-paper plan
Student model	None	Smoothed mastery per chapter, updated after every attempt
Paper format	Manual layout	Section A to D layout and PDF export generated automatically
Explainability	Not recorded	Every allocation has a written reason

4.3 INPUTS, PROCESSING AND OUTPUTS

Table 4.1: Inputs, processing and outputs of the system

Aspect	Definition
Users	Class 10 and Class 12 students, and tutors who review progress
Inputs	Subject, chapters to practise, total marks, difficulty preference (Easy, Medium, Hard, Mixed), and the marks the student scores for each question after attempting a paper
Processing	Chapter classification, weight calculation, mark allocation, question selection under an exact-total constraint, mastery update
Outputs	A formatted question paper (on screen and as PDF), a topic-wise performance report with solutions, and an updated mastery profile that shapes the next paper
Constraints	The total marks must equal the requested total exactly; sections follow the board pattern; recently used questions are avoided; the core logic must be deterministic and explainable

4.4 SUCCESS CRITERIA

The system is judged by the following measurable criteria, which are tested in Section 6:

1. A generated paper totals exactly the requested marks whenever a paper is produced; otherwise the student gets a clear message.
2. A weak chapter receives a larger share of marks than a strong chapter for the same student.
3. The mastery value of a chapter changes predictably after each attempt and the next paper

reflects that change.

4. The same inputs always give the same mark allocation; the project has an automated test suite for this behaviour.
5. A question that the student solved before is tagged in the paper.
6. When the AI is not available, the paper is still produced from the question bank and the student is told why.

CHAPTER 5
METHODOLOGY

5.1 SYSTEM ARCHITECTURE

The application has four layers (Figure 5.1). The student works in a web browser (presentation layer). Flask receives each request, finds the logged-in student and calls the services (application layer). The services are Python classes that hold all decisions: AdaptiveEngine, QuestionGenerator, PerformanceAnalyzer, LLMService and PDFGenerator (service layer). They read and write SQLite through SQLAlchemy models (data layer). Only LLMService talks to an external system, the Gemini API.

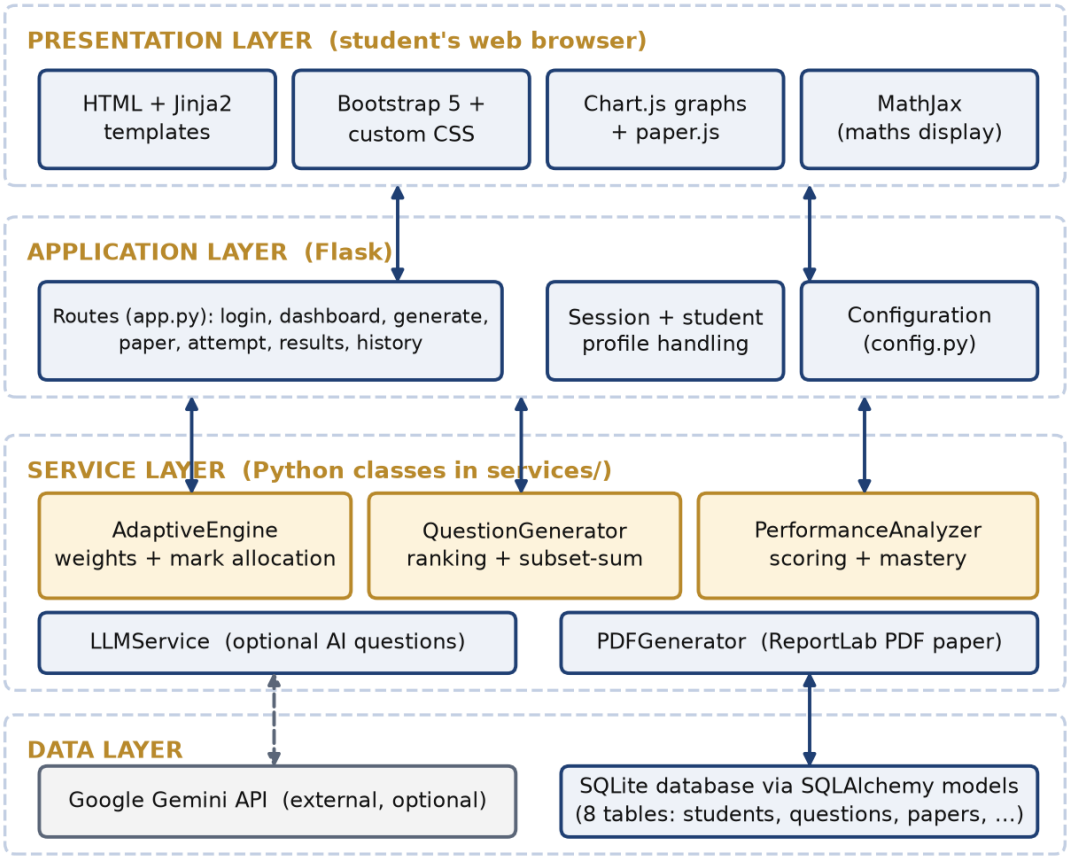


Figure 5.1: System architecture of the Adaptive Question Paper Generator

The behaviour of the system over time is a closed feedback loop (Figure 5.2). Marks entered after an attempt change the mastery of each chapter; the mastery changes the weights; the weights change how the marks of the next paper are shared; and the next paper produces new marks. The steps are:

1. the student attempts the paper and enters the marks for each question;
2. the marks are added up for each chapter and turned into a percentage;
3. the mastery of the chapter is updated;

4. the chapter is classified as Weak, Average or Strong;
5. a weight is assigned to the chapter;
6. the total marks are shared among the chapters;
7. questions are picked to match the marks, and questions solved before are tagged.

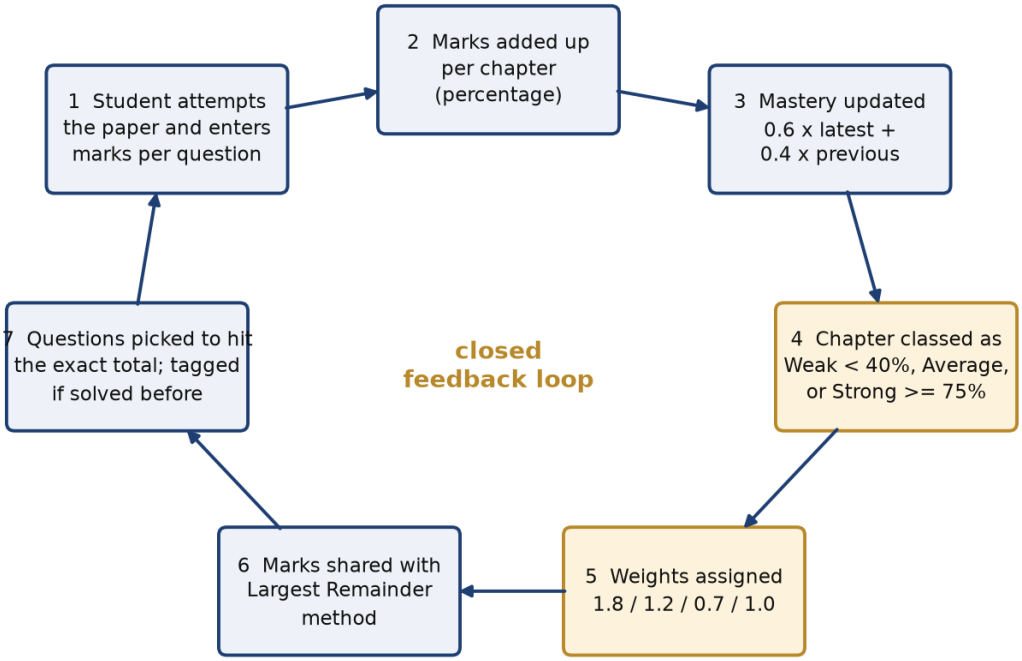


Figure 5.2: Closed feedback loop between attempts and the next paper

5.2 TOOLS AND TECHNOLOGIES

1 Hardware requirements

The application is light-weight. The following configuration is sufficient; the project was developed and tested on a Mac computer.

- **Operating system:** Windows 10 or later, macOS 12 or later, or a current Linux distribution.
- **CPU and RAM:** a dual-core 64-bit processor at 2 GHz or faster, and 4 GB of RAM (8 GB recommended).
- **Storage:** about 500 MB of free disk space for Python, the libraries and the project; the SQLite database is below 1 MB.
- **Other:** a current web browser; an internet connection for the interface libraries and, optionally, for the Gemini API; a printer for the PDF papers.

2 Software requirements

Table 5.1: Software components and their purpose

Component	Version / source	Purpose
Python	3.10 or later (tested on 3.12)	Language of the server and of all algorithms
Flask	3.x	Web framework: routes, sessions, Jinja2 templating
Flask-SQLAlchemy	3.1	Object-relational mapping of the eight database tables
SQLite	built into Python	Stores students, questions, papers, responses and mastery
ReportLab	current release	Board-format PDF with page numbering
Requests	current release	HTTP calls to the Gemini API
Bootstrap and Bootstrap Icons	5.3.3 / 1.11.3	Responsive interface and icons
Chart.js	4.4.1	Score-trend graph on the dashboard
MathJax	3.2.2	Displays mathematical expressions in AI-written questions
Google Gemini API	Interactions API	Optional AI writing of questions and grading of typed answers
pytest	8.x	Automated tests

Flask [5], [13] maps web addresses to Python functions, keeps the logged-in student in a session and fills HTML pages through the Jinja2 template engine. SQLAlchemy [6] lets each table be written as a Python class. ReportLab [7] draws the question paper as a PDF with a two-pass canvas so that every page can show "Page n of N". Chart.js [9] draws the score graph. The Gemini API [8] is optional: without an API key, or when the daily limit is reached, the application works from its own question bank.

5.3 ADAPTIVE MARK ALLOCATION

For every selected chapter the AdaptiveEngine reads the stored mastery score and gives the chapter a weight (Table 5.2). The thresholds and weights are configuration values in config.py.

Table 5.2: Chapter classification and weights

Chapter level	Mastery score	Weight	Purpose
Unattempted	no attempt yet	1.0	Balanced starting point
Weak	below 40%	1.8	Remediation: extra practice
Average	40% to 74%	1.2	Reinforcement
Strong	75% and above	0.7	Retention: revision, never zero

The total marks T are then shared among the n chapters with the Largest Remainder (Hare-Niemeyer) method [10] in three steps. First, every chapter receives a minimum of one mark when T is at least n. Second, the remaining T - n marks are split in proportion to the weights, so that chapter i gets (T - n) x w(i) / sum of w, and the whole part of each share is kept. Third, the marks still left over are handed out, one each, to the chapters with the largest fractional remainders. The result is whole marks that add up exactly to T, with no rounding drift. The engine also writes a plain-language reason for each chapter, shown to the student under "Why this split?".

5.4 QUESTION SELECTION

The QuestionGenerator selects the questions of a curriculum paper (Figure 5.3). The candidate questions of each chapter are first ranked with the scores of Table 5.3, so that questions that match the chosen difficulty come first and questions from the student's last three papers are pushed down. A small random jitter gives variety when the same paper is requested twice.

Table 5.3: Scores used to rank candidate questions

Rule	Score added
Difficulty equals the chosen preference (Easy, Medium or Hard)	+3.0
Difficulty one level away from the preference	+1.5 (+1.0 for Hard when Medium is chosen)
Difficulty two levels away from the preference	+0.5
Preference is Mixed	+2.0 for every question
Question appeared in the student's last three papers	-4.0
Random jitter	between 0.0 and 1.0

The target marks of each chapter are then met by a backtracking subset-sum search [11]: for each ranked question the search tries to include it, and if the running total overshoots or no

solution is found it removes the question and tries the next one, until a set of distinct questions with exactly the target marks is found. If a chapter cannot be matched exactly because of the mark sizes available, a global solver searches over all chapters together, requires the exact total and chooses the combination with the smallest squared deviation from the chapter targets. If even this fails, the student receives a clear message instead of a wrong paper. The chosen questions are finally sorted into Section A (1 mark), Section B (2 marks), Section C (3 marks) and Section D (4 or 5 marks), with multiple-choice questions first.

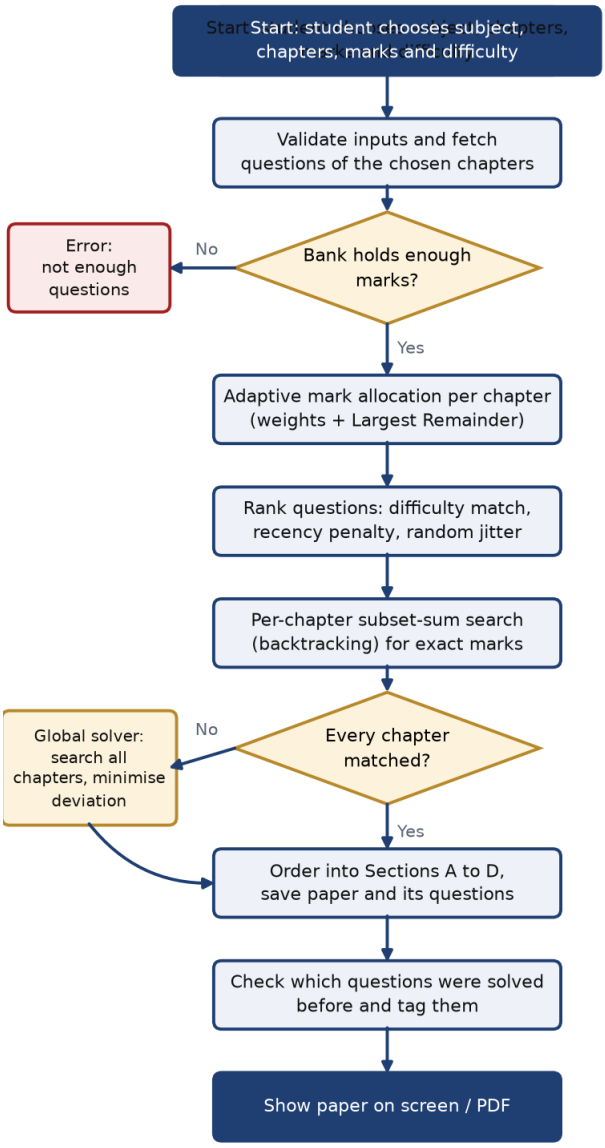


Figure 5.3: Flowchart of curriculum paper generation

5.5 PERFORMANCE TRACKING

When a student submits a paper, the PerformanceAnalyzer checks that every question has a mark between 0 and the marks allocated and that the paper was not submitted before, and then stores one response per question. The marks are added up for each chapter and turned into a

percentage. The mastery score of a chapter is the percentage itself at the first attempt. After that it is an exponential moving average [12]:

$$mastery = 0.6 \times latest\ percentage + 0.4 \times previous\ mastery$$

and the chapter is re-classified with the thresholds of Table 5.2.

The analyzer also finds the questions of a paper that the same student solved before. Only submitted papers of the same student count, the paper itself is excluded, and for a paper that is already submitted only papers submitted earlier count. The most recent earlier attempt supplies the date and the score shown in the "Previously solved" tag, which appears on the paper, the attempt page, the results page and the PDF.

5.6 AI QUESTION GENERATION

In AI mode the adaptive engine first decides how many marks each chapter receives, and a mix of 1, 2, 3 and 4-mark questions is planned for every chapter. One request to the Gemini API then asks for all the questions of the paper, stating the chapter, the type and the exact marks of each. The reply is cleaned of markdown fences and parsed as JSON; because mathematical text such as \frac contains backslashes that are not valid JSON, the parser repairs them. Every item is validated, and MCQ options are de-duplicated and limited to four. An item that is unusable is replaced by a question-bank question of the same chapter and marks. If the whole call fails (no key, daily limit, network problem, unusable reply) the paper is built from the question bank, and a message tells the student why. Temporary overload of the service is retried. One request per paper is used instead of one per question because of the daily limit of the free tier (Section 6.4).

5.7 DATABASE DESIGN

The relational schema is defined with SQLAlchemy and stored in SQLite. It has eight tables (Figure 5.4, Table 5.5). Papers keep the selected chapters, the target marks and the actual distribution as JSON text, together with the written rationale. Deleting a paper deletes its question links and responses, while a question that is part of a paper cannot be deleted by accident.

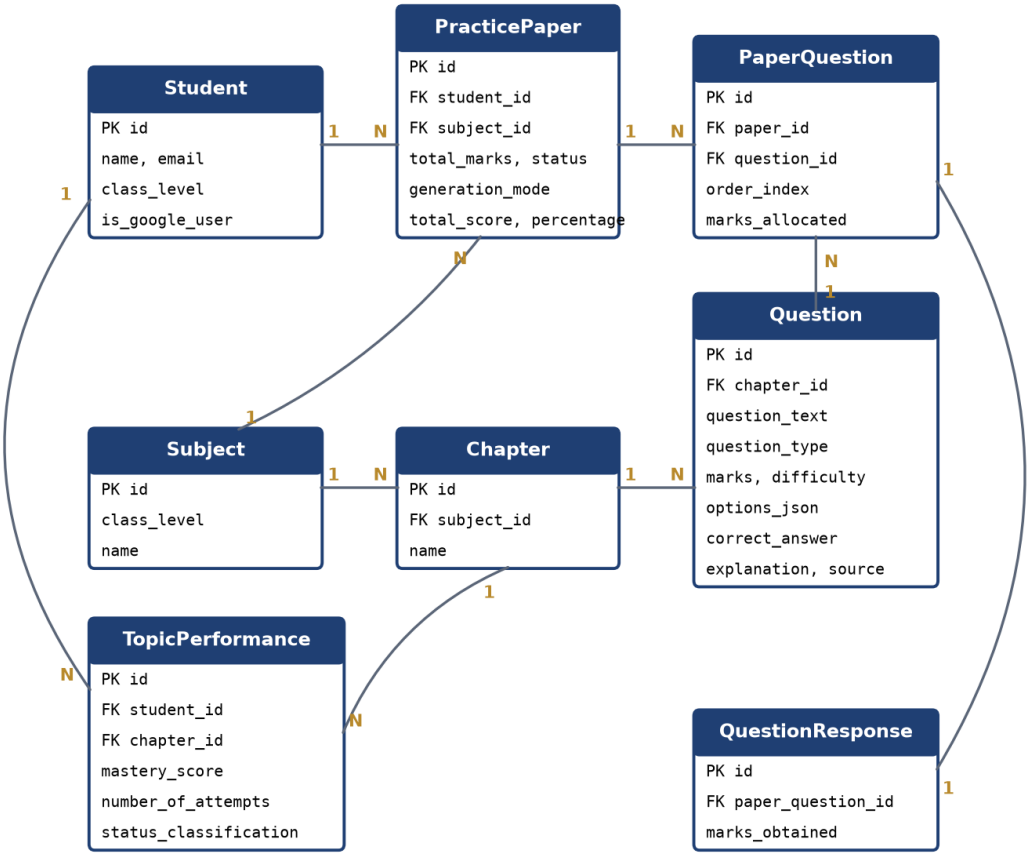


Figure 5.4: Entity-relationship diagram of the database

Table 5.5: Tables of the database

Table	Content
students	Name, e-mail, class level, Google profile data
subjects, chapters	Subjects of a class and the chapters of each subject
questions	Text, type (MCQ, short, long), marks, difficulty, options, answer, marking scheme, source (bank or AI)
practice_papers	One generated paper: student, subject, marks, mode, status, score, target and actual marks, rationale
paper_questions	Questions of a paper in order, with the marks allocated
question_responses	Marks obtained and the answer typed for each question of a submitted paper
topic_performances	Per student and chapter: latest percentage, mastery score, attempts, level

5.8 USER INTERFACE DESIGN

Flask routes connect the pages to the services (Table 5.4). The interface was designed for a student of about fifteen years: the dashboard opens with one "next step" suggestion, chapters are shown as cards with plain labels (Needs practice, Getting better, Strong, Not tried yet), technical words such as "weights" are avoided, the generate page is a three-step form, and a

"please wait" screen is shown while an AI paper is being written.

Table 5.4: Main routes of the web application

Route	Method	Purpose
/login, /auth/google, /logout	GET, POST	Choose a student profile, sign in with Google, sign out
/	GET	Dashboard: next step, chapter cards, score graph, questions to review
/generate	GET, POST	Three-step form; builds a curriculum paper or an AI paper
/paper/<id> and /paper/<id>/pdf	GET	Question paper on screen and as PDF
/attempt/<id>	GET, POST	Attempt page; submits marks and updates mastery
/results/<id>	GET	Score by chapter, next-paper plan, answers and solutions
/history	GET	List of all papers with filters
/question-bank, /question-bank/add	GET, POST	Browse and add questions
/api/preview-adaptive-allocation	POST	Live preview of the mark split on the generate page
/api/ai-generate-questions, /api/ai-evaluate-answer, /api/ai-save-generated-question	POST	Write questions with AI, grade a typed answer, save an AI question

5.9 TESTING METHOD

The rules are checked with automated pytest tests that run against an in-memory database. They cover the exactness of the mark allocation, the question selection, the mastery update, the detection of repeated questions, the PDF, every main route and the three outcomes of AI generation. The call to the Gemini API is replaced by a fake reply in the tests, so they need no key and use no quota. The AI mode was also tried with the real service, and the stored answers were checked by hand. The results are given in Section 6.

CHAPTER 6

RESULTS

6.1 ADAPTIVE MARK ALLOCATION

The allocation was checked with a student who attempts Class 10 Mathematics (Trigonometry, Statistics, Probability). With no history every chapter has weight 1.0 and receives 10 of 30 marks. In the first attempt the student scored 20% in Trigonometry, 60% in Statistics and 100% in Probability, so the chapters became Weak, Average and Strong. The next 30-mark paper gave 14, 10 and 6 marks (Table 6.1, Figure 6.1). The weak chapter received more than twice the marks of the strong chapter, the strong chapter was not dropped, and the total was exact. The same student received 9, 7 and 4 marks in a 20-mark paper and 24, 16 and 10 marks in a 50-mark paper. These numbers come from running the engine itself.

Table 6.1: Mark allocation in a 30-mark paper

Chapter	Score in attempt 1	Level	Weight	Marks in paper 2
Trigonometry	20%	Weak	1.8	14
Statistics	60%	Average	1.2	10
Probability	100%	Strong	0.7	6
Total			3.7	30

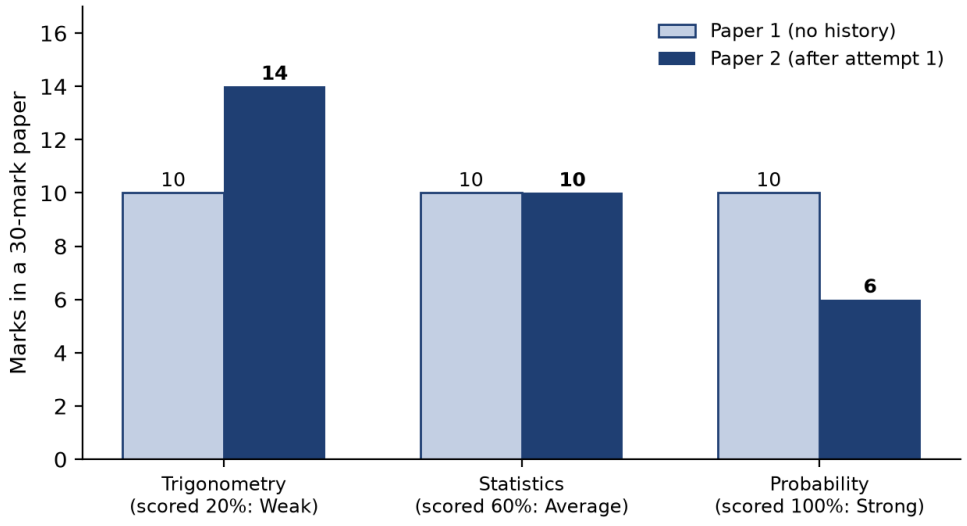


Figure 6.1: Marks given to each chapter before and after the first attempt

6.2 MASTERY TRACKING

For a chapter in which the student scores 20%, 60%, 80% and 50% in four attempts, the mastery score follows the rule of Section 5.5 and changes smoothly (Table 6.2, Figure 6.2). After

the second attempt the chapter moves from Weak to Average. The drop to 50% in the fourth attempt lowers the mastery only to 56.2, so a single poor result does not swing the next paper too far.

Table 6.2: Mastery after four attempts

Attempt	Score in attempt	Mastery	Level
1	20%	20.0	Weak
2	60%	44.0	Average
3	80%	65.6	Average
4	50%	56.2	Average

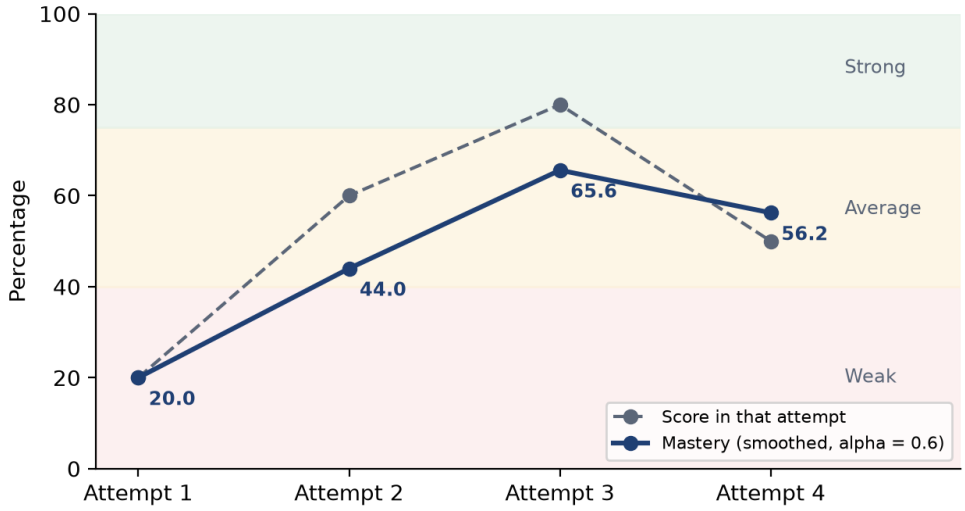


Figure 6.2: Mastery score compared with the score in each attempt

6.3 PREVIOUSLY SOLVED QUESTIONS

A student with three submitted papers was given a new 30-mark paper. Nine of its 14 questions had been solved before, and all nine carried the "Previously solved" tag with the date and score of the earlier attempt, together with a summary at the top of the paper ("9 of 14 questions already solved before") (Figure 6.6). The large number of repeats is caused by the small question bank: Class 10 Mathematics has only 48 questions and every 30-mark paper uses about 14 of them. Adding more questions is the remedy, and the tag makes the repetition visible and honest.

6.4 AI PAPER GENERATION

A first version of the AI mode made one request to the Gemini API for each question. A 20-mark paper needed about eight requests, and the free tier of the model used in the tests allows only 20 requests per day, so the limit was reached during testing. A test run took more than two minutes, and when requests failed the paper was silently filled with template questions that were not real. The mode was redesigned (Table 6.3): one request now writes the whole paper, failures

are retried when the service is only overloaded, an unusable question is replaced from the bank, and if the AI is not available at all the paper is made from the bank and the student is told why.

Table 6.3: AI paper generation: one request per question and one request per paper

Aspect	One request per question	One request per paper
Requests for a 20-mark paper	about 8	1
Time	more than 2 minutes in one test run	about 8 seconds (two test runs)
When the AI fails	template questions filled the paper silently	bank questions are used and the student is told
Marks in the prompt	not sent	chapter, type and exact marks of every question

With the new design a 20-mark Class 10 Mathematics paper was written in about 8 seconds, with eight real questions and exactly 20 marks, and none of the questions came from templates (Figure 6.9). The stored answers were checked by hand: for example "If 3 cos θ = 2 sin θ, find tan θ" is answered 3/2, the mean of the first five prime numbers is 5.6, the mode of 2, 3, 3, 4, 4, 4, 5, 5, 6 is 4, the probability of a number between 2 and 6 on a die is 1/2, and tan(A+B) = √3, tan(A-B) = 1/√3 gives A = 45° and B = 15°; all are correct. One defect was found and fixed: the model sometimes listed the right answer as a fifth, duplicate option of an MCQ, so options are now de-duplicated and limited to four. A Class 12 paper of 20 marks was also written in about 8 seconds through the web form, with seven questions and exactly 20 marks. Only the Class 10 answers listed above were checked by hand, so AI-written questions must be reviewed by a teacher before they are used in a class.

6.5 AUTOMATED TESTING

The project has 27 automated tests, which all pass in about 0.6 seconds (Table 6.4). They were repeated many times to make sure that none of them depends on chance.

Table 6.4: Automated tests

Test file	Tests	What it checks
test_adaptive_engine.py	3	Weights, exact integer allocation, explanations
test_question_generator.py	4	Exact total, distinct questions, sections, errors
test_performance_analyzer.py	3	Marks validation, mastery update, classification
test_previous_attempts.py	4	Repeated-question detection and tag on the paper page and PDF
test_ai_paper.py	4	One request per paper, fallback to the bank, replacement, option clean-up
test_pdf_generator.py	1	Valid PDF output
test_routes.py	8	Login, dashboard, generate, attempt, results, APIs, profile
Total	27	All passed

6.6 USER INTERFACE

The interface was rebuilt for a younger student. The login page (Figure 6.3) lets the student pick a profile. The dashboard (Figure 6.4) shows one next step, three numbers and a card for each chapter. The generate page (Figure 6.5) has three numbered steps. The paper (Figure 6.6) shows how the marks are split and tags the questions solved before. The attempt page (Figure 6.7) lets the student choose answers, see solutions and enter marks, and the results page (Figure 6.8) shows the score by chapter and what happens next. Figure 6.9 shows a paper written by the AI mode.

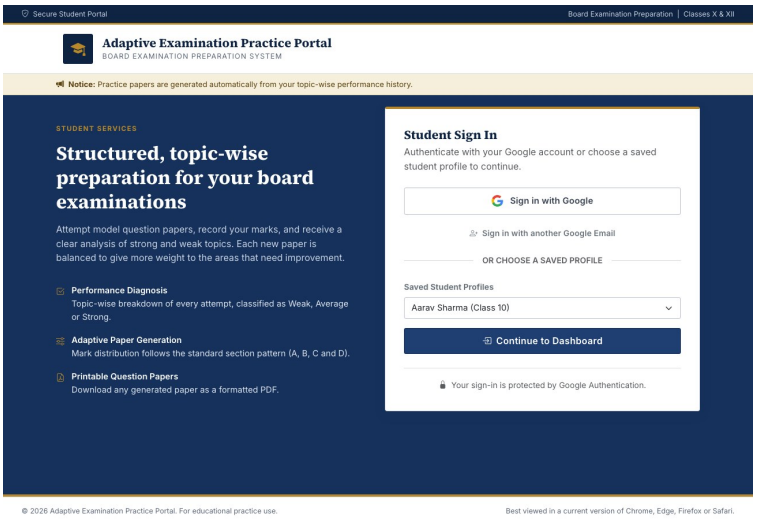


Figure 6.3: Login page

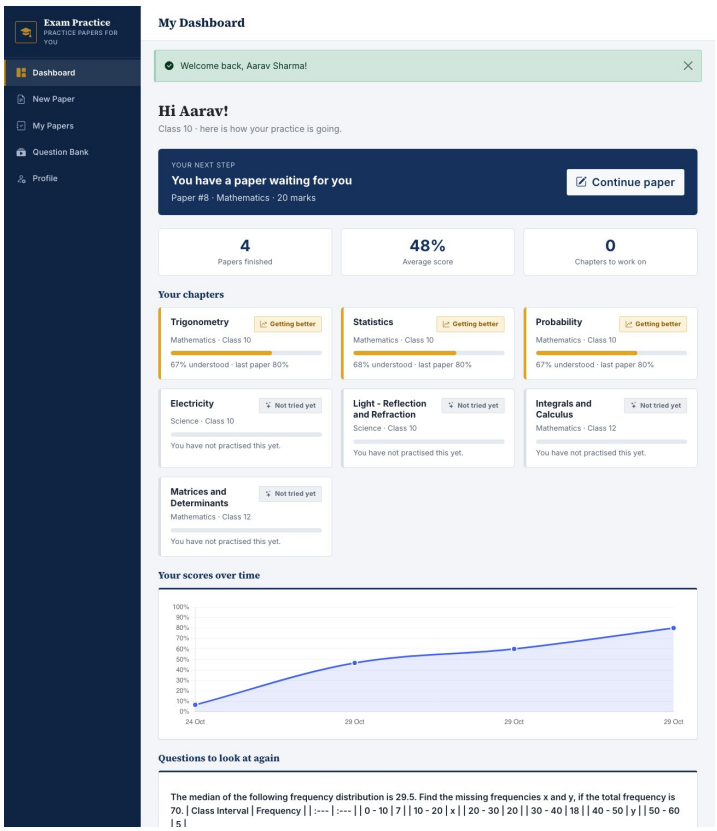


Figure 6.4: Student dashboard

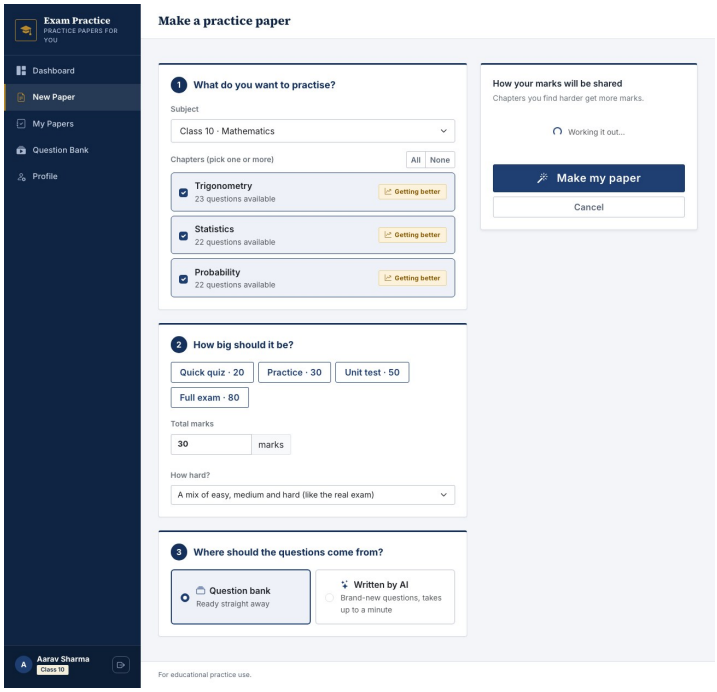


Figure 6.5: Make a practice paper (generate page)

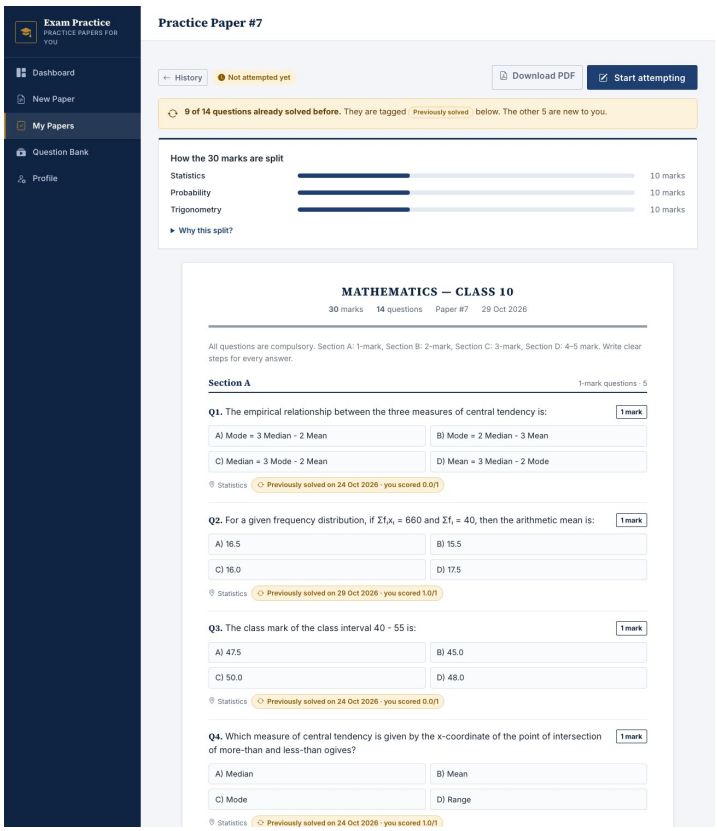


Figure 6.6: Question paper with "Previously solved" tags

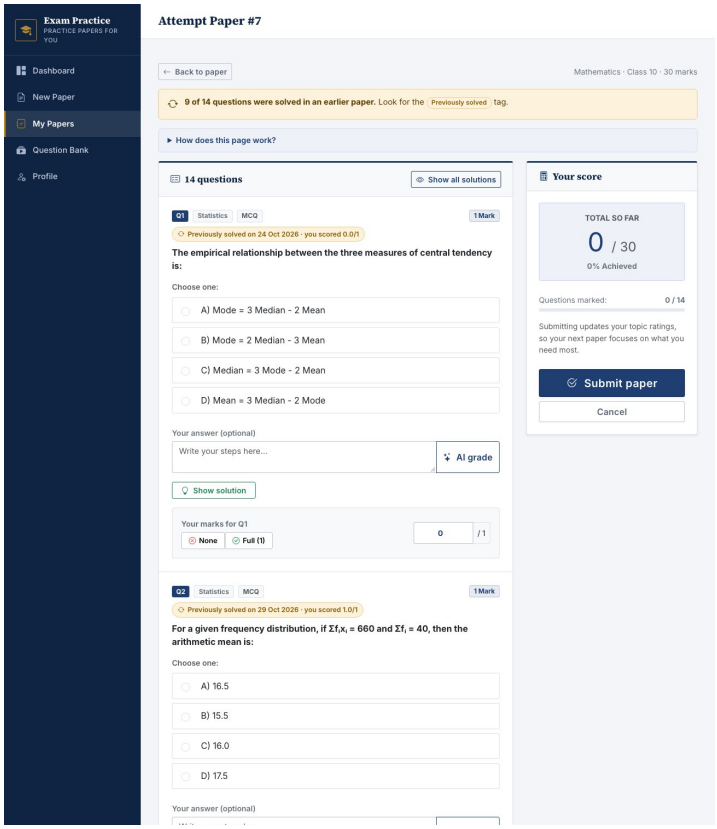


Figure 6.7: Attempt page with self-assessment

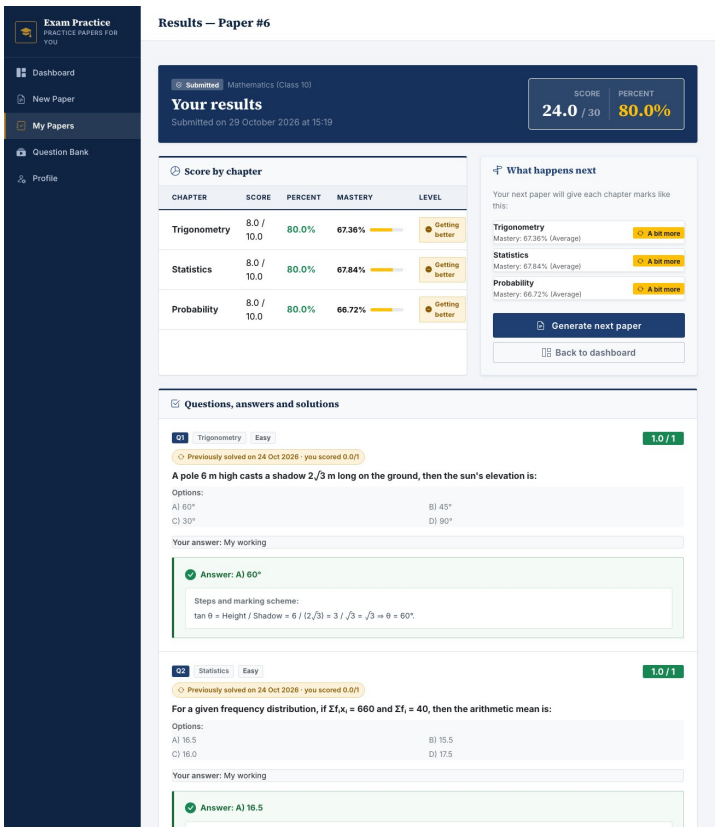


Figure 6.8: Results page with chapter scores and solutions

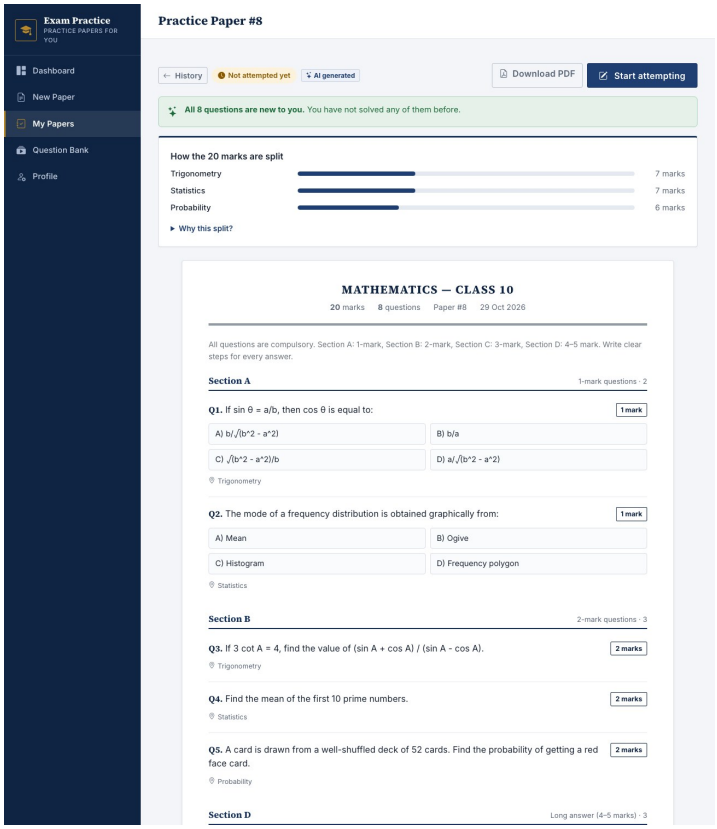


Figure 6.9: Paper written by the AI mode

6.7 DISCUSSION

The results meet the success criteria of Section 4.4. Marks are shared among the chapters by explainable rules, always add up exactly to the requested total, and react to the student's results:

in the worked example a weak chapter received 14 marks and a strong chapter 6. Mastery is smoothed so that one result does not dominate, questions the student has solved before are tagged, papers can be printed as PDF, and the optional AI mode writes new questions with a single request and falls back safely. The 27 automated tests pass.

The limitations of Section 2.4 remain: the question bank is small, marks are self-assessed, the AI-written questions need teacher review, and the effect on learning has not yet been measured in a classroom.

CHAPTER 7

FUTURE SCOPE

7.1 IMPROVEMENTS

- Grow the question bank across more subjects and chapters, reviewed by teachers, so that repeated questions become rare.
- Let a teacher review and approve AI-written questions before they enter the question bank.
- Verify the Google sign-in token, move to a production server and a larger database, and package the application in a container.
- Host the interface libraries locally so that the application works fully offline in a classroom.
- Run a pilot with a class and compare the improvement on weak chapters with a control group.

7.2 RESEARCH DIRECTIONS

- Estimate mastery more precisely with Item Response Theory or Bayesian Knowledge Tracing [2], [3], keeping the present rules as a baseline.
- Calibrate difficulty from real student results instead of fixed Easy, Medium and Hard tags.
- Detect near-duplicate questions with text embeddings, and tag new questions to chapters automatically.
- Read marks from scanned answer sheets with optical character recognition, so that marks are not typed by hand.
- Print every generated paper automatically to a classroom printer, and run the portal offline on low-cost classroom devices.

CHAPTER 8
REFERENCES

8.1 LIST OF JOURNALS

[1] Brusilovsky, P. "Methods and techniques of adaptive hypermedia." User Modeling and User-Adapted Interaction, 6(2-3), 87-129, 1996.

[2] Corbett, A. T. and Anderson, J. R. "Knowledge tracing: Modeling the acquisition of procedural knowledge." User Modeling and User-Adapted Interaction, 4(4), 253-278, 1994.

[3] Pelanek, R. "Bayesian knowledge tracing, logistic models, and beyond: an overview of learner modeling techniques." User Modeling and User-Adapted Interaction, 27(3-5), 313-350, 2017.

[4] Kurdi, G., Leo, J., Parsia, B., Sattler, U. and Al-Emari, S. "A systematic review of automatic question generation for educational purposes." International Journal of Artificial Intelligence in Education, 30(1), 121-204, 2020.

8.2 LIST OF WEBSITES (URLs)

[5] Flask documentation. Source: https://flask.palletsprojects.com/

[6] SQLAlchemy documentation. Source: https://docs.sqlalchemy.org/

[7] ReportLab documentation. Source: https://docs.reportlab.com/

[8] Google Gemini API documentation. Source: https://ai.google.dev/gemini-api/docs

[9] Chart.js documentation. Source: https://www.chartjs.org/docs/

[10] Largest remainder method. Source: https://en.wikipedia.org/wiki/Largest_remainder_method

8.3 LIST OF BOOKS

[11] Cormen T. H., Leiserson C. E., Rivest R. L., Stein C. Introduction to Algorithms. 3rd ed. MIT Press; 2009.

[12] Hyndman R. J., Athanasopoulos G. Forecasting: Principles and Practice. 3rd ed. OTexts; 2021.

[13] Grinberg M. Flask Web Development: Developing Web Applications with Python. 2nd ed. O'Reilly Media; 2018.

[14] van der Linden W. J. Linear Models for Optimal Test Design. Springer; 2005.